

Technical Documentation: N-body simulation

Miłosz Grześkowiak

March 2026

Abstract

Simulating the dynamics of N-body gravitational systems is a computationally intensive problem fundamental to celestial mechanics and astrophysics. This document details the architecture, mathematical foundation, and implementation of a high-performance, interactive N-body simulation software developed in C++. The computational core leverages the Eigen library to handle vectorized linear algebra operations and the numerical integration of Newtonian equations of motion, ensuring mathematical stability and efficiency. To facilitate real-time visual analysis of the orbital dynamics, the software integrates OpenGL for hardware-accelerated 3D rendering. This rendering pipeline is managed by a responsive graphical user interface built with the Qt framework, allowing users to dynamically adjust initial conditions, integration parameters, and viewing angles. Ultimately, this project provides a robust, cross-platform tool that bridges accurate numerical physics with interactive visualization, making it suitable for both educational demonstration and the exploration of complex orbital mechanics.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Project Scope	3
1.2.1	Functional Capabilities	3
1.2.2	Physical and Mathematical Assumptions	3
1.3	Technology Stack and Architecture	4
2	Mathematical Model	5
2.1	Governing Equations	5
2.2	System State	5
3	Numerical Implementation	5
3.1	Integration Method	5
3.2	Time step	6
3.3	Handling singularities	6
4	Software Architecture	6
4.1	Overall Design	6
4.1.1	High-Level Architecture (MVC Pattern)	6
4.1.2	Main Subsystems	7
4.2	Eigen integration	7
4.2.1	Vectorized Physics Operations	7
4.2.2	Memory Alignment and Data Layout	7
4.3	Complexity	7
5	Visualization and User Interface	7
5.1	The Qt GUI	7
5.1.1	The OpenGL Bridge (QOpenGLWidget)	8
5.1.2	Signals and Slots Mechanism	8
5.2	The OpenGL pipeline	8
5.2.1	Buffer Management (VBO & VAO)	8
5.2.2	Shader Architecture	9
6	User Guide	9
6.1	Compilation	9
6.2	Running the code	9
7	Validation and Benchmarks	9

1 Introduction

1.1 Motivation

The primary motivation for this project is to bridge the theoretical principles of classical mechanics with the practical paradigms of modern software engineering. As a student of both Physics and Applied Informatics, I designed this project to demonstrate how computational techniques can be leveraged to solve complex, non-linear physical problems that defy analytical solutions.

In physics, the N -body problem—predicting the individual motions of a group of celestial objects interacting with each other gravitationally—is a foundational challenge. For any system where $N \geq 3$, the system exhibits chaotic behavior and cannot be solved with closed-form mathematical equations. Therefore, understanding these systems requires heavy reliance on numerical simulations.

From an Applied Informatics perspective, simulating an N -body system presents an excellent architectural and algorithmic challenge. It requires synthesizing multiple domains of computer science:

- **High-Performance Computing:** Efficiently calculating the $\mathcal{O}(N^2)$ gravitational interactions requires a performant language (C++) and highly optimized linear algebra operations (Eigen).[1]
- **Computer Graphics:** Translating the raw numerical data into a comprehensible format requires real-time, hardware-accelerated rendering pipelines (OpenGL).
- **Software Architecture and UI/UX:** Providing users with the ability to dynamically alter the simulation (e.g., changing time steps, adjusting masses) requires a robust, event-driven graphical user interface (Qt) safely separated from the physics calculation thread.

Ultimately, this project serves as a capstone that unites my studies. It showcases that a deep understanding of physical laws is necessary to build an accurate simulation, but a rigorous application of informatics is required to make that simulation computationally feasible, stable, and interactive.

1.2 Project Scope

The objective of this project is to deliver a standalone, interactive desktop application capable of simulating and visualizing the gravitational interactions of N celestial bodies in three-dimensional space.

To achieve a balance between physical accuracy and computational real-time performance, the scope of the project is defined by the following boundaries and features:

1.2.1 Functional Capabilities

- **Numerical Simulation:** The core engine computes the continuous state vectors (position, velocity, and acceleration) of N massive particles over discrete time steps Δt , utilizing the Eigen library for optimized linear algebra operations.[1]
- **Real-Time Visualization:** The system state is rendered in real-time utilizing an OpenGL 3D graphics pipeline, allowing the user to visually track orbital trajectories.
- **Interactive GUI:** A graphical user interface developed with the Qt framework provides dynamic control over the simulation. Users can pause/resume time, modify the integration step size, and manipulate camera perspectives.

1.2.2 Physical and Mathematical Assumptions

- **Classical Mechanics:** The simulation operates strictly within the domain of Newtonian physics. General relativistic effects (such as time dilation or frame-dragging) are considered out of scope.

- **Point Masses:** All celestial bodies are treated as point masses. Rigid body dynamics, rotational inertia, and tidal forces are excluded from the mathematical model.
- **Collision Handling:** To prevent mathematical singularities when the distance between two bodies approaches zero ($|\mathbf{r}_j - \mathbf{r}_i| \rightarrow 0$), a softening parameter ϵ is introduced into the force calculation. Physical collisions and mass-merging mechanics are out of scope.[1]

1.3 Technology Stack and Architecture

To achieve both high-performance numerical computation and real-time interactive visualization, the software architecture is built upon a carefully selected stack of industry-standard libraries. The application strictly adheres to the separation of concerns, decoupling the physics engine from the graphical user interface.

- **C++ (Core Language):** Chosen for its zero-overhead abstractions, deterministic memory management (RAII), and raw execution speed. Given that the direct-summation N -body problem has a computational complexity of $\mathcal{O}(N^2)$, a compiled, highly optimized language is strictly required to simulate a meaningful number of bodies at interactive framerates.
- **Eigen (Linear Algebra Library):** Eigen is a C++ template library utilized for all vector and matrix operations within the physics engine. It was selected for its heavy use of template metaprogramming and lazy evaluation, which prevents the creation of temporary objects during complex mathematical expressions. Furthermore, Eigen automatically leverages SIMD (Single Instruction, Multiple Data) instructions, significantly accelerating the 3D spatial calculations (e.g., using `Eigen::Vector3d` for position, velocity, and force vectors).
- **OpenGL (Graphics API):** Responsible for the real-time 3D rendering of the simulation. By utilizing the programmable shader pipeline, the graphical representation of the celestial bodies is offloaded entirely to the GPU. This ensures that the rendering process does not bottleneck the CPU, which must remain dedicated to the intensive numerical integration of the physical system.
- **Qt Framework (GUI and Application Loop):** Qt serves as the backbone for the application’s window management, event loop, and user interface. It provides the `QOpenGLWidget`, which seamlessly bridges the OpenGL rendering context with the desktop window. Additionally, Qt’s Signals and Slots mechanism is used to safely handle asynchronous user inputs (such as adjusting the simulation time step or pausing the integration) without introducing race conditions.
- **GTest (Unit Tests):** Utilized as the primary C++ unit testing and mocking framework to ensure the numerical accuracy and stability of the simulation’s core logic. It allows for the isolation of individual physics components—such as the N -body integrator and collision detection algorithms—from the UI and rendering pipelines. By employing **Test Fixtures**, the project maintains consistent environments for verifying $\mathcal{O}(N^2)$ complexity optimizations and Eigen-based vector math. Furthermore, **GMock** is leveraged to simulate peripheral hardware or asynchronous GUI events, ensuring that the multithreaded synchronization logic (Signals and Slots) remains robust under edge-case scenarios.
- **Multithreading (Concurrency):** To prevent the UI from freezing during heavy physics calculations, the architecture employs multithreading (e.g., utilizing `OpenMP` or `QThread`). The numerical integration loop runs on a dedicated worker thread, continuously updating the state vectors, while the main thread handles GUI events and triggers OpenGL draw calls based on the most recent, mutex-protected physics state.
- **CMake (Build System):** The project utilizes CMake as its meta-build system, ensuring that the software is cross-platform and can be easily compiled on different operating systems while managing the complex linkage between Qt, OpenGL, and Eigen.

2 Mathematical Model

The simulation is based on the classical Newtonian N-body problem, which describes the motion of a group of particles interacting through gravitational forces. Each particle is treated as a point mass, and the evolution of the system is governed by a set of second-order ordinary differential equations (ODEs).

2.1 Governing Equations

The fundamental interaction between two bodies, i and j , is governed by **Newton’s Law of Universal Gravitation**. For a system of N bodies, the total force $\mathbf{F}_i$ acting on body i is the vector sum of the gravitational pull from all other $N - 1$ bodies:

$$\mathbf{F}_i = \sum_{j=1, j \neq i}^N G \frac{m_i m_j}{\|\mathbf{r}_j - \mathbf{r}_i\|^2} \frac{\mathbf{r}_j - \mathbf{r}_i}{\|\mathbf{r}_j - \mathbf{r}_i\|} \quad (1)$$

By applying **Newton’s Second Law** ($\mathbf{F} = m\mathbf{a}$), we derive the acceleration $\mathbf{a}_i$ of body i :

$$\mathbf{a}_i = \frac{d^2 \mathbf{r}_i}{dt^2} = \sum_{j=1, j \neq i}^N G \frac{m_j (\mathbf{r}_j - \mathbf{r}_i)}{\|\mathbf{r}_j - \mathbf{r}_i\|^3} \quad (2)$$

2.2 System State

The state of the entire simulation at any discrete time t is represented by a high-dimensional state vector. In the implementation, this is mapped to **Eigen::Vector3d** structures for computational efficiency.

The state of an individual body i is defined by:

- **Mass** (m_i): A scalar constant representing the gravitational "charge" of the body.
- **Position** ($\mathbf{r}_i$): A 3D vector $[x, y, z]^T$ in Cartesian coordinates.
- **Velocity** ($\mathbf{v}_i$): A 3D vector representing the first derivative of position with respect to time, $\frac{d\mathbf{r}_i}{dt}$.

The **Global System State** is the union of all individual states:

$$\mathbf{S}(t) = \{(m_1, \mathbf{r}_1, \mathbf{v}_1), (m_2, \mathbf{r}_2, \mathbf{v}_2), \dots, (m_N, \mathbf{r}_N, \mathbf{v}_N)\} \quad (3)$$

In terms of the software architecture, this state is protected by **mutexes** to ensure that the integrator thread can update the state while the OpenGL thread performs a consistent read for rendering.

3 Numerical Implementation

While the governing equations provide a continuous mathematical description, the simulation requires a discrete approximation to evolve the system state over time.

3.1 Integration Method

The project implements the **Velocity Verlet algorithm**. This method is preferred in molecular dynamics and celestial mechanics due to its **time-reversibility** and **symplectic nature**, which ensures that the long-term energy error of the N-body system remains bounded rather than drifting indefinitely.

The state update for each body i at time $t + \Delta t$ is calculated in three distinct stages:

1. Intermediate Position Update:

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \mathbf{v}_i(t)\Delta t + \frac{1}{2}\mathbf{a}_i(t)\Delta t^2 \quad (4)$$

2. Intermediate Velocity (Half-step):

$$\mathbf{v}_i(t + \frac{1}{2}\Delta t) = \mathbf{v}_i(t) + \frac{1}{2}\mathbf{a}_i(t)\Delta t \quad (5)$$

3. Final Velocity (Full-step):

After computing the new acceleration $\mathbf{a}_i(t + \Delta t)$ using the updated positions:

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t + \frac{1}{2}\Delta t) + \frac{1}{2}\mathbf{a}_i(t + \Delta t)\Delta t \quad (6)$$

Implementation Note: This approach requires storing the acceleration from the previous step to compute the new position, effectively doubling the precision of the trajectory compared to first-order methods.

3.2 Time step

To ensure a stable and deterministic simulation, a constant **time step** (Δt) is utilized.

- **Decoupled Physics:** The integration frequency is independent of the GUI refresh rate. By fixing Δt , we prevent "numerical jitters" that occur when frame rates fluctuate, ensuring the physics remains consistent regardless of hardware performance.
- **Upper Bound:** The value of Δt is constrained by the expected velocity of the fastest-moving body to prevent "tunnelling" or massive energy spikes during close-range interactions. **(TO BE IMPLEMENTED)**

3.3 Handling singularities

To address the $\frac{1}{r^2}$ singularity in the gravitational potential, the implementation uses **Plummer Softening**. This replaces the point-mass singularity with a softened density profile, effectively treating bodies as having a finite spatial "blur."

The acceleration $\mathbf{a}_i$ is computed using:

$$\mathbf{a}_i = \sum_{j \neq i} \frac{Gm_j(\mathbf{r}_j - \mathbf{r}_i)}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \epsilon^2)^{\frac{3}{2}}} \quad (7)$$

Where ϵ is the softening length. This ensures that the force remains finite even when $\|\mathbf{r}_j - \mathbf{r}_i\| \rightarrow 0$, preventing the Velocity Verlet algorithm from encountering infinite accelerations that would otherwise crash the simulation.

4 Software Architecture

4.1 Overall Design

4.1.1 High-Level Architecture (MVC Pattern)

The project architecture relies on a decoupled adaptation of the Model-View-Controller (MVC) paradigm:

- **Model (Logic and Data):** Encapsulated within the `GravityEngine` and `CelestialBody` classes. This layer manages the global system state, physical properties (masses, velocity vectors), and trajectory histories. It operates entirely independently of the graphical interface.

- **View (Presentation):** Composed of two distinct layers. The first is the standard user interface built with Qt widgets (`MainWindow`). The second is the hardware-accelerated 3D viewport rendered via OpenGL (`NBodyWidget`).
- **Controller (Routing):** Qt’s **Signals and Slots** mechanism serves as the controller, capturing asynchronous user inputs (button clicks, slider adjustments) and translating them into thread-safe commands for the physics engine or the OpenGL state machine.

4.1.2 Main Subsystems

The application is divided into three primary engines operating in tandem:

4.2 Eigen integration

The project integrates the **Eigen** template library to handle all high-dimensional linear algebra operations. Eigen was selected to provide a high-level abstraction for vector math without sacrificing the performance required for $\mathcal{O}(N^2)$ calculations.

4.2.1 Vectorized Physics Operations

The core of the N-body simulation relies on calculating distance vectors, magnitudes, and force summations. Eigen enables these operations through **SIMD (Single Instruction, Multiple Data)** instructions (SSE/AVX), which allow the CPU to process multiple floating-point operations in a single clock cycle.

- **Expression Templates:** By using Eigen’s expression templates, complex operations like $\mathbf{r}_{ij} = \mathbf{r}_j - \mathbf{r}_i$ are evaluated lazily. This prevents the creation of unnecessary temporary objects in memory, which is critical when performing $N(N - 1)$ calculations per time step.

4.2.2 Memory Alignment and Data Layout

To ensure maximum throughput between the **Numerical Implementation** and the **Graphics Pipeline**, data is stored in a contiguous memory layout.

- **Array of Structures (AoS) vs. Structure of Arrays (SoA):** While Eigen facilitates both, the implementation utilizes a contiguous array of `Eigen::Vector3` objects. This layout is cache-friendly for the Velocity Verlet integrator and allows for a direct memory map to the `QOpenGLBuffer`.
- **Alignment:** While the current implementation relies on standard memory alignment provided by the C++17 runtime, the architecture is designed to support **SIMD (Single Instruction, Multiple Data)** vectorization. Future iterations may utilize `Eigen::aligned_allocator` to ensure 16 or 32-byte alignment, further optimizing the force calculation loops for AVX/SSE instructions.

4.3 Complexity

5 Visualization and User Interface

5.1 The Qt GUI

The simulation utilizes the **Qt Framework** as its foundational backbone. Qt is responsible for managing the application’s main event loop, providing cross-platform window management, and handling asynchronous user inputs without blocking the high-intensity mathematical computations.

5.1.1 The OpenGL Bridge (QOpenGLWidget)

Qt serves as the host environment for the hardware-accelerated graphics pipeline via the `QOpenGLWidget` class. This wrapper abstracts away the platform-specific windowing APIs (such as WGL on Windows or GLX on Linux) and provides a safe, object-oriented context for executing OpenGL commands. The rendering lifecycle is tightly controlled by overriding three core virtual functions:

- **initializeGL():** Executed once upon context creation. Used to compile GLSL shader programs, configure depth testing, and allocate the initial **Vertex Buffer Objects (VBOs)** on the GPU.
- **resizeGL(int w, int h):** Automatically triggered during window resize events. It recalculates the projection matrix to maintain the correct aspect ratio of the 3D viewport.
- **paintGL():** The core rendering function. It maps the latest Eigen state vector to the GPU buffers and issues the `glDrawArrays` command.

5.1.2 Signals and Slots Mechanism

To facilitate safe, asynchronous communication between the isolated components, the project leverages Qt's **Signals and Slots** paradigm.

This mechanism is primarily utilized for:

- **Simulation Control:** Safely transmitting user commands (such as Play, Pause, or Step) from the UI buttons to the physics thread.
- **Parameter Adjustment:** Allowing real-time modification of simulation variables, such as the time step (Δt) or the Plummer softening factor (ϵ), without requiring an application restart.
- **Camera Navigation:** Intercepting mouse and keyboard events (`mouseMoveEvent`, `wheelEvent`) to manipulate the View matrix (panning, zooming, and rotating the simulation space).

5.2 The OpenGL pipeline

The simulation utilizes the **OpenGL Programmable Pipeline** to achieve high-performance 3D visualization. By leveraging a hardware-accelerated approach, the CPU is responsible only for calculating the physics state, while the graphical representation—including coordinate transformations and point-sprite generation—is offloaded to the GPU.

5.2.1 Buffer Management (VBO & VAO)

To minimize the overhead of CPU-to-GPU data transfers, the engine employs a buffered memory strategy using **Vertex Buffer Objects (VBO)** and **Vertex Array Objects (VAO)**.

- **Vertex Buffer Objects (VBO):** These represent blocks of raw memory allocated on the GPU's VRAM. In each frame, the updated position vectors ($\mathbf{r}_i$) from the physics engine are "streamed" to the VBO using `QOpenGLBuffer`. This ensures that the per-particle data is already on the graphics card before the draw call is issued.
- **Vertex Array Objects (VAO):** The VAO acts as a state container that stores the format of the vertex data. It defines the layout of the VBO (e.g., stride and offset for position and color attributes). By binding a single VAO, the application restores the entire vertex configuration state in one call, significantly reducing driver overhead during the main loop.

5.2.2 Shader Architecture

The project utilizes custom **GLSL (OpenGL Shading Language)** programs to process the N-body data:

- **Vertex Shader:** Performs the projection of 3D world coordinates into 2D clip space. It utilizes uniform matrices (Projection and View) to handle camera movement and zooming handled by the Qt GUI.
- **Fragment Shader:** Responsible for the final pixel color. It can be configured to color-code particles based on their velocity magnitude ($||\mathbf{v}_i||$) or mass (m_i), providing visual feedback on the system's kinetic energy distribution.

Performance Note: By using `glDrawArrays`, the GPU renders all N bodies in a single batch. This prevents the "draw-call bottleneck" that would occur if each celestial body were rendered individually.

6 User Guide

6.1 Compilation

6.2 Running the code

7 Validation and Benchmarks

References

- [1] Sverre J Aarseth. *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge University Press, 2003.